

```
1  #%%
2  # SCOPE
3
4  def f1():
5      x =10
6      print('sono f1 ->',x)
7      return None
8
9  #-----
10 f1()
11 print('sono nel prg->',x)
12
13
14 #%%
15 # SCOPE
16
17 def f1():
18     x =10
19     print('sono f1 ->',x)
20     return None
21
22 #-----
23 x=100
24 print('sono nel prg->',x) # 100
25 f1() #10
26 print('sono nel prg->',x)# 100
27
28 #%%
29
30
31 # SCOPE
32
33 def f1():
34     print('sono f1 ->',x)
35     return None
36
37 #-----
38 x=100
39 print('sono nel prg->',x) # 100
40 f1() #100
41 x=200
42
43 print('sono nel prg->',x)# 100
44 f1() #200
```

```
45 #%%
46
47 #%%
48 # 7 maggio
49
50 # esempi ERRATI
51
52 def f():
53     print('CIAO')
54 # -----
55 print(f())
56
57 a= f() #None
58 print(a)
59
60
61 #%%
62
63 # SI
64
65 def doppio(x):
66     return 2*x
67
68 ###
69
70 a= doppio(10)
71 print(a)
72 print(doppio(2))
73
74
75 #%%
76 # più di un parametro
77
78 def area_rettangolo(l1, l2):
79     return l1*l2
80
81 def volume_parallelepipedo(l1, l2, l3):
82     return l1*l2*l3
83
84 #-----
85
86 # se restituisce 1 valore
87 # y = f(x)
88 # f(x) # il valore restituito viene perso
```

```
89
90
91 vc=volume_parallelepipedo(10,20,30)
92 print(vc)
93
94 #%%
95
96 #%%
97
98 def caratteristiche_geometriche(l1, l2):
99     """ questa funzione rende perimetro e area di un
    rettangolo
100         si richiama in questo modo:
101         v = caratteristiche_geometriche(l1, l2)
102         dove v è una tupla che contiene perimetro e area
103
104         oppure
105         v1 , v2 = caratteristiche_geometriche(l1, l2)
106         dove v1 è perimetro e v2 area
107
108     """
109     a= l1*l2
110     p = 2*(l1+l2)
111     return p, a
112
113
114 # se restituisce 2 valori
115 # f(x) # i valori restituiti vengono persi
116 # y1 = f(x) # i valori restituiti vengono memorizzati
    nella TUPLA y1
117 # y1, y2 = f(x) # i valori restituiti vengono memorizzati
    in y1, y2
118
119 # se restituisce 3 valori
120 # y1, y2 = f(x) # ERRORE
121
122 perimetro, area = caratteristiche_geometriche(1,2)
123
124 print(perimetro, area)
125 v = caratteristiche_geometriche(10, 12)
126 print(v)
127 #%%
128 help (caratteristiche_geometriche)
129 #%%
```

```
130 l=[1,2]
131 help(l.append)
132 ###
133
134 ###
135 # SCOPE
136
137 def funzione():
138     x = 10
139     print('Dentro:', x)
140 #-----
141 funzione()
142 print('Fuori:', x)    # ERRORE
143 ###
144
145 ###
146 def funzione():
147     x = 10
148     print('Dentro:', x)
149 #-----
150 x=5
151 funzione()
152 print('Fuori:', x)    #
153 ###
154
155 ###
156 # la f. ha 2 modi per creare var locali:
157
158 def f(x):
159     # parametro è var locale
160     y =10 #creo var locale
161     return 2*x*y
162
163
164 ###
165 #ERRORE IMPORTANTE
166
167 def doppio():
168     return 2*x
169
170 #----
171
172 x=10
173 y = doppio()
```

```
174
175 print(x)
176 print(y)
177
178
179 #%%
180
181 #ERRORE IMPORTANTE
182
183 def media():
184     s=0
185     for v in voti:
186         s=s+v
187     return s/len(voti)
188
189 voto=[30, 30]
190
191 m= media()
192 print(m)
193
194
195
196
197
198 #%%
199
200 #%%
201 def f():
202     x=20
203     print('dentro',x)
204
205 # - -
206
207 x=10
208 print(x)
209 f()
210 print(x)
211
212
213
214 #%%
215
216 #%%
217 #ERRORE
```

```
218 def f():
219     print('dentro1',x)
220     x= x+1
221     print('dentro2',x)
222 #--
223
224 x=10
225 print(x) # 10
226 f() # 10, 11 --- errore?
227 print(x) #10


---


228 ###
229 #ERRORE
230 def f():
231     # print('dentro1',x)
232     x= x+1
233     print('dentro2',x)
234 #--
235
236 x=10
237 print(x) # 10
238 f() # 10, 11 --- errore?
239 print(x) #10


---


240 ###
241 #SI
242
243
244 def f():
245     # print('dentro1',x)
246     y= x+1
247     print('dentro2',x, y)
248 #--
249
250 x=10
251 print(x) # 10
252 f() # 10, 11 --- errore?
253 print(x) #10


---


254 ###
255 # prg non accede a var della funzione
256
257 def f():
258     ww=10
259
260 #
261 y =5
```

```

262 f()
263 print(wv)
264
265
266
267 #%%
268 # RIPRENDIAMO SCOPE DA QUI
269 #%%
270 # giocatore 1 e giocatore 2+ altre 10 persone
271 # basket -> squadre da 6
272 # prob che g1 e g2 siano nella stessa squadra?
273 # -> (1/6)*(1/11) - > 1.5%
274 # -> 50%
275 #-> 25%
276
277 #f che mi rende 1 se i giocatori sono nella stessa squadra
    :
278 import random
279
280 def insieme(n):
281     """ rende 1 se i giocatori 1, 2 sono nella stessa
    squadra"""
282     s1=[]
283     s2=[]
284
285     giocatori= list(range(1,n+1))
286     # for g in range(1,n+1))
287     for g in giocatori:
288         if len(s1) == n/2:
289             s2.append(g)
290         elif len(s2) == n/2:
291             s1.append(g)
292         else:
293             #tiro la moneta
294             v=random.randint(1,2)
295             if v == 1:
296                 s1.append(g)
297             else:
298                 s2.append(g)
299     #print(s1, s2)
300     # if (1 in s1)== (2 in s1)
301     if ((1 in s1) and (2 in s1) ) or ((1 in s2) and (2 in
    s2) ):
302         return 1

```

```
303     else:
304         return 0
305
306
307 # -----
308 insieme(12)
309
310
311
312 #%%
313 n_prove = 10000
314 conta = 0
315 for i in range(n_prove):
316     v = insieme(12)
317     conta=conta+v
318 prb = conta/n_prove
319 print(prb)
320
321
322
323 #%%
324
325 #11 maggio
326
327 def f():
328     ...
329
330 def f(x):
331     ...
332
333 def f()
334     ...
335     return y
336
337 def f(x):
338     ...
339     return y
340
341
342
343
344 #%%
345 def f(x):
346     y = x+1 #11
```



```
347     print('f', y) #11
348     return y ##11
349
350 ## ----
351 y =10
352 print(y) # 10
353 f(10)
354 print(y) #10
355
356 print('---')
357
358
359
360 #%%
361
362 #%%
363 # ok
364 def media(lista):
365     s=0
366     for el in lista:
367         s=s+el
368     return s/len(lista)
369
370 #-----
371 lista=[1,2,3,4,5,6,7,8,9,10]
372 lista1= [1,2]
373 m = media(lista1)
374 print(m)
375
376
377
378 #%%
379 # ERRORE
380 def media():
381     s=0
382     for el in lista:
383         s=s+el
384     return s/len(lista)
385
386 #-----
387 lista=[1,2,3,4,5,6,7,8,9,10]
388 lista1= [1,2]
389 m = media()
390 print(m)
```

```
391
392 ###
393 # errore grave
394 # leggere (e peggio ancora scrivere) var. globali dalla
funzione
395 def f(x, y):
396     lista[0] = 100
397     z = x+y
398     return z
399
400 #-----
401
402 lista = [1,2,3,4,5,6,7,8,9,10]
403 print(lista)
404 a = 10
405 b = 20
406 c = f(a,b)
407 print(c)
408 print(lista)
409
410
411
412 ###
413 # errore - effetto collaterale
414
415 def f(lista):
416     lista[0] = 0
417     print(lista) #0
418
419 #----
420 l1 = [1,2,3]
421 f(l1)
422 print(l1) # 1,2,3
423
424
425
426
427 ###
428 # esempio di effetto collaterale:
429 # funzione che somma solo elementi pari
430 # ok
431 def f1(lista):
432     s = 0
433     i=0
```

```

434     while i<len(lista):
435         if lista[i]%2 == 0:
436             # pari
437             s=s+lista[i]
438             i = i+1
439     return s
440
441 # alternative
442 # for el in lista...
443 # for i in range(len(lista))...
444
445 l1=[1,2,3,4]
446 ris= f1(l1)
447 print(ris)
448 print(l1)


---


449 ###
450 # esempio di effetto collaterale:
451 # funzione che somma solo elementi pari
452 # risolvo l'esercizio in modo volutamente più complicato
    del dovuto
453 # per mostrare effetti cpllaterali
454
455 # 1- scorro la lista e metto a 0 elementi dispari
456 # 2- sommo tutto
457
458
459 def f2(lista):
460     i = 0
461     while i<len(lista):
462         if lista[i]%2 != 0:
463             # dispari
464             lista[i]=0
465             i = i+1
466         # gli el dispari sono 0
467         #ora sommo
468         s = 0
469         for el in lista:
470             s=s+el
471     return s
472
473 # alternative
474 # for el in lista...
475 # for i in range(len(lista))...
476

```

```

477 l1=[1,2,3,4]
478 ris = f2(l1)
479 print(ris)
480 print(l1)
481 #%%
482 # effetti collaterali VOLUTI:
483 # print
484 # append
485
486 l1=[1,10, 2,4]
487
488 l1.sort() #effetto collaterale VOLUTO
489 print('l1',l1)
490 print('----')
491
492 l2=[1,10, 2,4]
493 l_ordinata = sorted(l2) # NO effetti collaterali
494 print('l_ordinata',l_ordinata)
495 print('l2',l2)
496
497
498 #%%
499
500 #----
501 # parametri ingresso
502
503
504 def f_media(valori, pesi):
505     numeratore =0
506     denominatore =0
507     for i in range (len(valori)):
508         v = valori[i]
509         p = pesi [i]
510         numeratore = numeratore + v*p
511     for i in range (len(pesi)):
512         p = pesi [i]
513         denominatore = denominatore + p
514     return numeratore/denominatore
515
516 voti =[30, 20]
517 crediti=[3, 10]
518 mp= f_media(voti, crediti)
519 print(mp)
520

```

```

521 #%%
522 # f che calcola media pesata se metto i pesi.,
523 # altrimenti media semplice
524
525 # soluzione: paraemtri opzionali
526 # la media semplice si può calcolare mettendo pesi pari a
    1
527 # e calcolando la media pesata
528 def f_media(valori, pesi = None ):
529     n = len(valori)
530     if pesi is None:
531         # media semplice
532         #pesi = [1,1,1,1,1,1]
533         pesi=[1]* n
534
535     numeratore =0
536     denominatore =0
537     for i in range (n):
538         v = valori[i]
539         p = pesi [i]
540         numeratore = numeratore + v*p
541     for i in range (n):
542         p = pesi [i]
543         denominatore = denominatore + p
544     return numeratore/denominatore
545
546 voti =[30, 20]
547 crediti=[3, 10]
548 # MEDIA PESATA
549 mp= f_media(voti, crediti)
550 print(mp)
551 # MEDIA ARITMETICA
552 mp= f_media(voti)
553 print(mp)
554 #%%
555
556 #%%
557
558 #%% md
559 # 13 maggio 2026
560 #%%
561
562 #%%
563 import random

```

```

564 #random.seed(67)
565 a = random.randint(1,10)
566 b = random.randint(1,10)
567 c = random.randint(1,10)
568
569 print(a, b,c)
570
571 ###
572 # Squadra basket
573
574 import random
575 # random.randint(1,2)
576 def insieme(n):
577     """ rende 1 se i due amici 1 e 2 sono nella stessa
squadra"""
578     s1=[]
579     s2=[]
580     giocatori = list(range(1, n+1))
581     #for g in range(1, n+1)
582     random.shuffle(giocatori)
583     for g in giocatori:
584         if len(s1)== n/2:
585             s2.append(g)
586         elif len(s2)== n/2:
587             s1.append(g)
588         else:
589             # tiro moneta - 1,2
590             v = random.randint(1, 2)
591             if v==1:
592                 s1.append(g)
593             else:
594                 s2.append(g)
595     #print(s1)
596     #print(s2)
597     if ((1 in s1) and (2 in s1)) or ((1 in s2) and (2 in
s2)):
598         # 1 e 2 sono in s1?
599         # oppure
600         # 1 e 2 sono in s2?
601         # cond alternativa
602         # if (1 in s1) == (2 in s2)
603         return 1
604     else:
605         return 0

```

```
606 # ATTENZIONE c'è un ERRORE
607
608 #%%
609 n_trial = 100000
610 contatore = 0
611 for i in range(n_trial):
612     r = insieme(12)
613     contatore += r # contatore = contatore + r
614
615 prob = contatore/ n_trial
616
617 print(prob)
618 #%%
619
620 #%%
621
622 #%%
623
624 #%%
625
626 #%%
627 # 14 maggio 2026
628 #%%
629
630 #%%
631 import random
632 def stima_area_cerchio(r, N = 1000):
633     # conto punti all'interno del cerchio
634     interni = 0
635     for _ in range (N):
636         # coordinate casuali nel quadrato
637         x = random.uniform(-r, r)
638         y = random.uniform(-r, r)
639         if (x**2 + y**2) < r**2:
640             interni += 1
641     area_stimata = 4* r**2 * (interni/N)
642     return area_stimata
643
644 #----
645 area_stimata = stima_area_cerchio(1, 1000000)
646 print(area_stimata)
647
648
649 #%%
```

```
650
651 ###
652 # bambini dispettosi
653
654 def azione_bambini(n):
655     """ dato il numero di piani,
656     rende la lista con secchi pieni (1) e vuoti (0)
657     """
658     lista_secchi = [0] * (n+1)
659     for i_bambino in range(1, n+1):
660         for i_p in range(i_bambino, n+1, i_bambino):
661
662             if lista_secchi[i_p]==1:
663                 lista_secchi[i_p]=0
664             else:
665                 lista_secchi[i_p]=1
666     return lista_secchi
667
668 lista_secchi = azione_bambini(6)
669 print(lista_secchi[1:])
670
671
672 ###
673
674 ###
675
676 ###
677
678 ###
679 # 18 maggio
680 ###
681
682 ###
683
684 ###
685 def somma(lista):
686     s=0
687     for el in lista:
688         s+=el
689     return s
690 def media(lista):
691     s= somma(lista)
692     m = s /len(lista)
693     return m
```



```
694
695
696 ###
697
698 ###
699 #21 maggio
700
701 ###
702
703 ###
704 # Ricerca binaria - versione elementare
705 # La lista deve essere ordinata
706
707
708
709 def ricerca_binaria(lista, valore):
710     sinistra = 0
711     destra = len(lista) - 1
712
713     while sinistra <= destra:
714         medio = (sinistra + destra) // 2
715
716         # Controlla il valore centrale
717         if lista[medio] == valore:
718             return medio # posizione trovata
719
720         # Cerca nella metà destra
721         elif lista[medio] < valore:
722             sinistra = medio + 1
723
724         # Cerca nella metà sinistra
725         else:
726             destra = medio - 1
727
728     return -1 # valore non trovato
729
730
731 # Esempio
732 numeri = [3, 7, 12, 18, 25, 31, 42]
733
734 x = 25
735
736 posizione = ricerca_binaria(numeri, x)
737
```

```
738 if posizione != -1:
739     print("Valore trovato in posizione:", posizione)
740 else:
741     print("Valore non trovato")
742 #%%
743
744 #%%
745
746 #%%
747
748 #%%
749
750 #%%
751
752 #%%
753
754 #%%
755
756 def merge(l1, l2):
757     l3 = []
758     i1 = 0
759     i2 = 0
760     while i1 < len(l1) and i2 < len(l2):
761         if l1[i1] < l2[i2]:
762             l3.append(l1[i1])
763             i1 += 1
764         else:
765             l3.append(l2[i2])
766             i2 += 1
767
768     if i1 == len(l1):
769         # l1 è finita prima
770         l3.extend(l2[i2:])
771     else:
772         l3.extend(l1[i1:])
773
774     return l3
775
776 #-----
777 lista1 = [3, 4, 6, 10]
778 lista2 = [2, 8, 16, 18, 20]
779 # [2, 3, 4, 6, 8, 10]
780 lista3 = merge(lista1, lista2)
781 print(lista3)
```

```
782
783
784
785 #%%
786
787
788 #%%
789
790 #%%
791
792 #%%
793
794
795
796
797 #%%
798
799 #%%
800
801 #%%
802
803 #%%
804
805 #%%
806 # 25 maggio
807
808 #%%
809 3>2
810 #%%
811 "ab">"ac"
812 #%%
813 z1= {'r':10, 'i':5}
814 z2= {'r':3, 'i':-1}
815 z3= {'r':-4, 'i':7}
816
817 lista=[z1, z2, z3]
818
819 print(lista)
820
821 #%%
822
823 #%%
824 import random
825 def selection_sort(lista):
```

```
826     n = len(lista)
827     for i in range(n):
828         idx_min=i
829         for j in range(i+1, n):
830             if lista[j]<lista[idx_min]:
831                 idx_min = j
832                 # metto il min nel punto giusto - scambio
833                 tmp =lista[i]
834                 lista[i] = lista[idx_min]
835                 lista[idx_min] = tmp
836
837
838 # ----
839 a = list(range(20))
840 print(a)
841
842 ###
843
844 ###
845
846 ###
847
848 ###
849 def f(a):
850     a=10
851
852
853 x = 20
854 f(x)
855 print(x) # 20
856
857
858
859 ###
860 def f(a):
861     # a=3
862     a[0]=30
863 ---
864
865 x = [0,1,2]
866 f(x)
867 print(x) #
868
869 ###
```

```
870 def f(x):
871     print(x)
872
873 ###
874 x=40
875 y=-1
876 f(x)
877 f(8)
878 f(y)
879 ###
880
881 # Selection Sort
882 import random
883 def selection_sort(lista):
884     n = len(lista)
885
886     for i in range(n):
887         # indice del valore minimo
888         idx_minimo = i
889
890         # cerca il minimo nella parte restante
891         for j in range(i + 1, n):
892             if lista[j] < lista[idx_minimo]:
893                 idx_minimo = j
894
895         # scambia gli elementi
896
897         tmp = lista[idx_minimo]
898         lista[idx_minimo] = lista[i]
899         lista[i] = tmp
900         # miglioramento: scambia solo se diversi
901         # if idx_minimo != i:...
902 #-----
903 # Esempio
904 numeri = list(range(20))
905 random.shuffle(numeri)
906 print(numeri)
907
908 selection_sort(numeri)
909 print(numeri)
910 # NON
911 # numeri = selection_sort(numeri)
912
913
```

```
914 #print("Lista ordinata:", ordinata)
915
916
917
918
919
920
921
922 #%%
923
924 # Selection Sort CRESCENTE
925 import random
926 def selection_sort(lista):
927     n = len(lista)
928
929     for i in range(n):
930         # indice del valore minimo (o massimo)
931         idx = i
932         # cerca il minimo nella parte restante
933         for j in range(i + 1, n):
934             if lista[j] > lista[idx]:
935                 idx = j
936
937         # scambia gli elementi
938
939         tmp = lista[idx]
940         lista[idx] = lista[i]
941         lista[i] = tmp
942 numeri = list(range(20))
943 random.shuffle(numeri)
944 print(numeri)
945
946 selection_sort(numeri)
947 print(numeri)
948 #%%
949
950
951
952 # Selection Sort CRESCENTE
953 import random
954 def selection_sort(lista, controllo = 0):
955     n = len(lista)
956
957     for i in range(n):
```

```
958         # indice del valore minimo (o massimo)
959         idx = i
960         # cerca il minimo nella parte restante
961         for j in range(i + 1, n):
962             if controllo==0:
963                 if lista[j] > lista[idx]:
964                     idx = j
965             else:
966                 if lista[j] < lista[idx]:
967                     idx = j
968
969
970         # scambia gli elementi
971
972         tmp = lista[idx]
973         lista[idx] = lista[i]
974         lista[i] = tmp
975
976     controllo=1
977     numeri = list(range(20))
978     random.shuffle(numeri)
979     print(numeri)
980
981     selection_sort(numeri, controllo)
982     print(numeri)
```